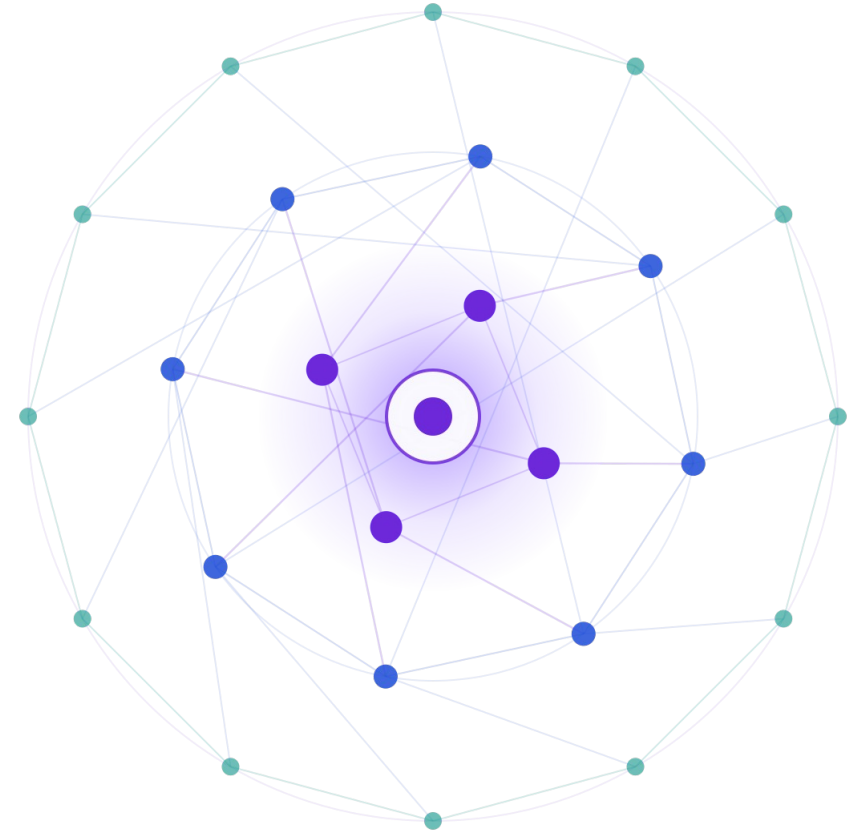


# Generative AI Hackathon

**3rd Edition** 2026 – 27

- Theme ID - Theme 2 · Smart Guided Troubleshooting Engine
- Team Name - Syntax Error
- College Name - M S Ramaiah Institute of Technology, Bengaluru
- L Srishant Reddy – 1ms25ci070@msrit.edu
- B Prajwal – 1ms25cs034@msrit.edu
- B Darshan – 1ms25ad008@msrit.edu
- M Kashif – 1ms25ci072@msrit.edu
- Submission Github link - <https://github.com/Srishant14/samsung-prism-troubleshooting-engine>



<42 ms • Type-safe • Zero-token

# Theme

THEME 2

## Smart Guided Troubleshooting Engine

Turn a plain-language device problem into a verified, step-by-step fix — instantly.

- Understand the problem the way users actually say it
- Decide deterministically — no free-text generation
- Guide with verified steps; escalate to a store or email when needed

**<42 ms**

end-to-end latency

**20×**

faster than conventional LLM-based solutions

**1,420**

tokens saved per session

# Existing Solutions & Gaps

## Generative LLM chatbots

Generate a fresh answer for every single query.

### THE GAP

- ~840 ms round-trip per session
- Token cost on every turn
- Free-text steps can hallucinate

## Static FAQs & keyword search

Match keywords against long help articles.

### THE GAP

- Miss plain-language phrasing
- Users dig through articles themselves
- No guided, step-by-step flow

## Vanilla RAG

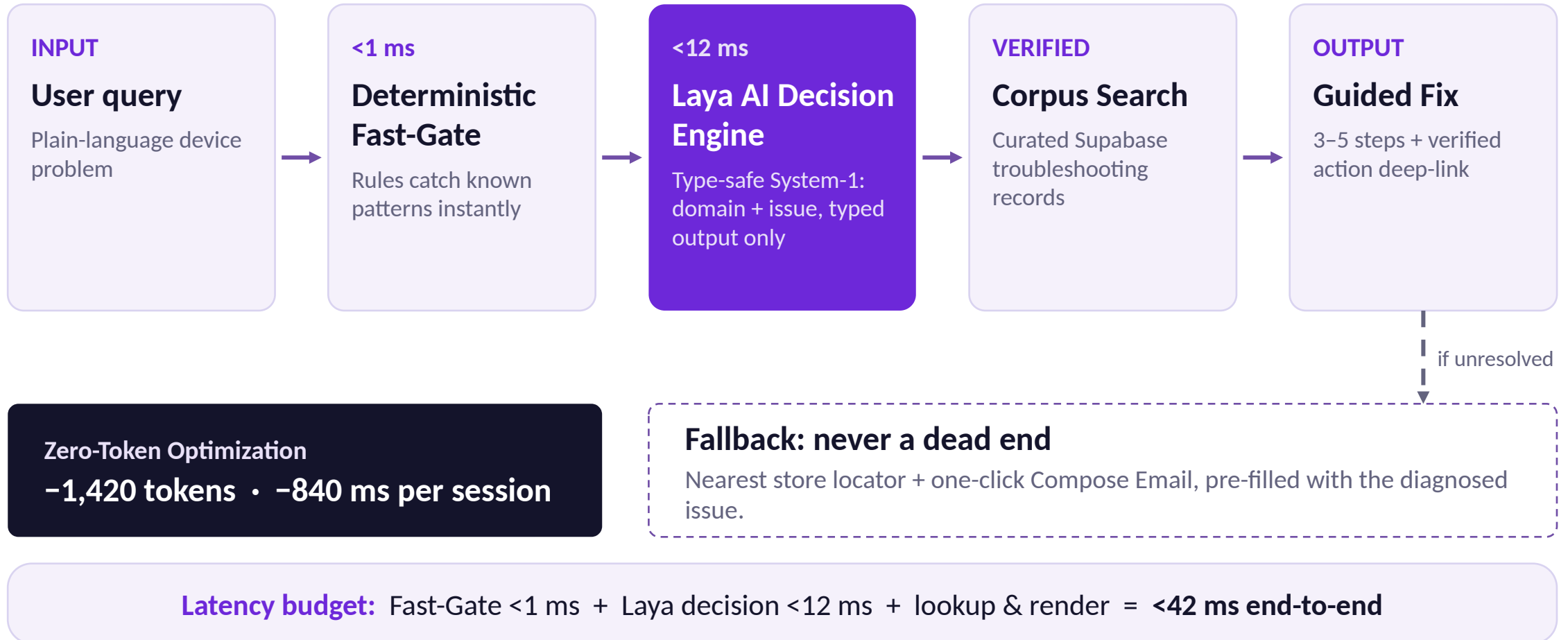
Embed, retrieve, then generate with an LLM.

### THE GAP

- Still pays the LLM generation cost
- Quality hinges on embeddings
- No deterministic fast path for common issues

**The gap:** nothing is fast, deterministic, and always gives the user a next step. We built exactly that.

# Our Solutions & Architecture Diagram



# Demo & Product Walkthrough

1

## Describe

Type the problem in plain words — no menus, no forms.

2

## Decide

Fast-Gate + Laya classify domain and issue in <12 ms.

3

## Guide

3–5 verified steps plus a one-tap action button.

4

## Escalate

Still stuck? Nearest store locator and a pre-drafted email.

SAMPLE RUN (illustrative)

**“My phone battery drains overnight”** → Laya decision: Battery › Fast drain  
→ Verified steps + battery-settings deep-link → **<42 ms**

# Tools and tech stack used

## DECISION ENGINE

### Laya AI v0.3.20

Type-safe System-1 engine (convaiinnovations/laya). Typed categorical output, no free text.

## RETRIEVAL

### Hybrid RAG pipeline

Deterministic fast-gate plus verified corpus search over curated records.

## DATA

### Supabase PostgreSQL

Curated troubleshooting table: Battery, Display, Camera, Performance.

## BACKEND

### Python + FastAPI

Lightweight API with Pydantic validation end to end.

## FRONTEND

### React + Vite + Tailwind

Instant, responsive guided-steps interface.

## BUILD

### Antigravity

AI-assisted implementation to ship fast.

# Impact & Use case

**20×**

faster than conventional LLM-based solutions

**840 ms**

latency saved per session

**1,420**

tokens saved per session

## At scale: 1M sessions (projected)

- 1.42 billion tokens saved
- ~233 hours of user waiting removed
- Sub-50 ms responses feel instant

## Use cases

- Battery drain and charging issues
- Display flicker and screen problems
- Camera faults
- Slow performance and lag
- Hardware fault? Nearest store + email

# Innovation highlights, results and limitations

## Innovation highlights

- Type-safe System-1 decision engine instead of a generative LLM
- 3-tier hybrid RAG: fast-gate → decision → verified search
- Zero-token hot path
- Always a next step: store locator + compose email

## Results

Conventional LLM-based ~840 ms



Syntax Error pipeline <42 ms



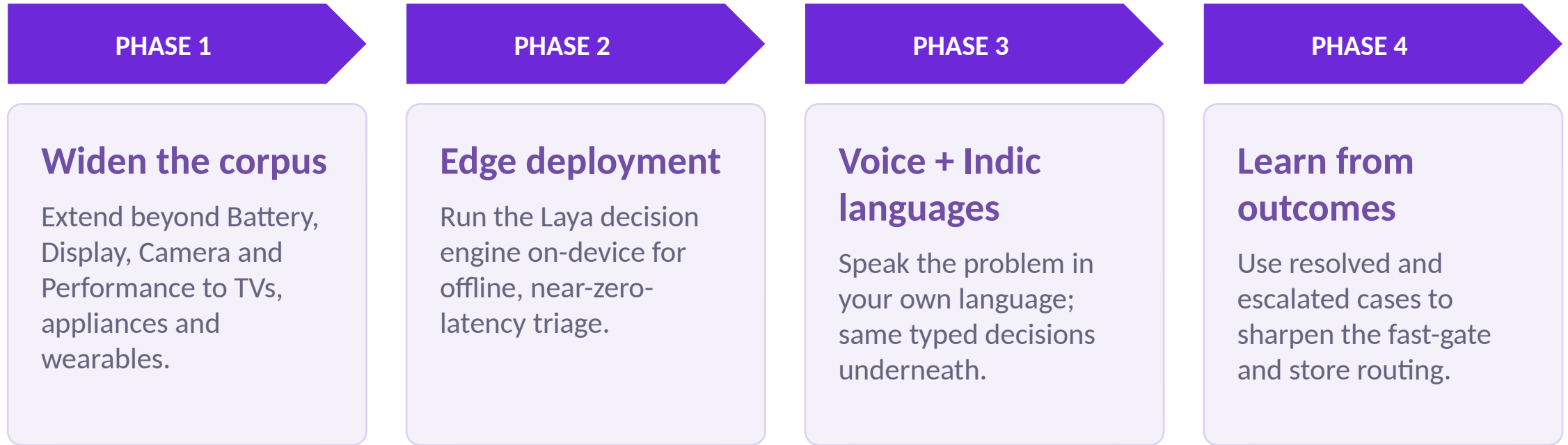
**20× faster**

-1,420 tokens · -840 ms per session

## Limitations (v1)

- Curated knowledge base of ~20–30 records across 4 domains
- Categorical decisions only — out-of-corpus issues go to the fallback
- Laya is early-stage (v0.3.x)
- Benchmarks from our prototype; to be validated at scale

# What's next



**Vision:** a <42 ms decision layer any device-support flow can plug into.

# Brownie points slide( differentiation)

## Type-safe by design

Typed primitives; categorical output only.

## Zero-token hot path

Saves 1,420 tokens per session.

## <42 ms end-to-end

20× faster than LLM-based flows.

## Hybrid 3-tier RAG

Fast-gate, AI arbitration, verified search.

## Verified answers only

Steps come from a curated corpus.

## Never a dead end

Store locator + compose email fallback.

|           | LLM-based       | Ours             |
|-----------|-----------------|------------------|
| Latency   | ~840 ms         | <42 ms           |
| Tokens    | Full generation | 1,420 saved      |
| Output    | Free text       | Typed categories |
| Decisions | Vary run to run | Deterministic    |
| Fallback  | Often none      | Store + email    |

# Checklist- Updated on Public GitHub

- Working prototype code — public or shared GitHub repo (Y) ✓
- README with reproducible setup instructions (Y) ✓
- Demo video, max 5 minutes (YouTube or Drive link) ✓
- Presentation file (PPT or PDF) (Y) ✓

# Thank you

**Team Syntax Error** · Theme 2 · <42 ms. Type-safe. Zero-token.